

Recap 2

Serializzazione con JSON

La **serializzazione** consente di convertire strutture dati Python in un formato testuale standard. **JSON** è il formato più comune per lo scambio dati tra servizi web perché è leggibile, leggero e compatibile con altri linguaggi.

In Python il modulo `json` integrato permette di trasformare oggetti complessi (come dizionari o liste) in stringhe, e viceversa, usando funzioni semplici.

```
import json
data = {"nome": "Alice", "eta": 30}
# Serializza in stringa JSON
json_str = json.dumps(data)
# Deserializza di nuovo in Python
print(json.loads(json_str)) # {'nome': 'Alice', 'eta': 30}
```

Serializzazione con YAML

YAML è un formato alternativo a JSON, più leggibile e adatto ai file di configurazione. Supporta commenti e indentazione intuitiva.

Con la libreria **PyYAML**, è possibile convertire strutture Python in YAML e viceversa.

```
import yaml
config = {"server": "localhost", "porta": 8080}
yaml_str = yaml.dump(config)
print(yaml.safe_load(yaml_str)) # {'server': 'localhost', 'porta': 8080}
```

Serializzazione Binaria con Pickle

Il modulo **pickle** consente di salvare oggetti Python in formato binario, mantenendone tipo e struttura.

È utile per il **salvataggio di stato** o il caching, ma va usato solo con dati fidati per motivi di sicurezza.

```
import pickle
dati = {"x": 42, "lista": [1, 2, 3]}
blob = pickle.dumps(dati)
print(pickle.loads(blob)) # {'x': 42, 'lista': [1, 2, 3]}
```

Interazione con Database SQLite

SQLite è un database **embedded** incluso in Python, perfetto per piccoli progetti o test locali.

Si interagisce con SQL tramite il modulo `sqlite3`, creando connessioni, eseguendo query e gestendo transazioni.

```
import sqlite3
conn = sqlite3.connect("db.sqlite")
cur = conn.cursor()
cur.execute("CREATE TABLE IF NOT EXISTS utenti (id INTEGER, nome TEXT)")
cur.execute("INSERT INTO utenti VALUES (?, ?)", (1, "Mario"))
conn.commit()
```

Lettura da SQLite

Le query di selezione (`SELECT`) restituiscono tuple Python. È possibile scorrerle con `fetchall()` o iterando sul cursore.

Ogni riga è rappresentata come una tupla di valori ordinati.

```
cur.execute("SELECT * FROM utenti")
for riga in cur.fetchall():
    print(riga)
conn.close()
```

Interazione con PostgreSQL

Con **PostgreSQL**, si lavora in modalità client-server. La libreria `psycopg2` fornisce un'interfaccia standard per SQL.

Si definiscono connessioni, cursori e query analoghe a SQLite, ma con parametri di rete.

```
import psycopg2
conn = psycopg2.connect(host="localhost", database="test", user="user",
password="pwd")
cur = conn.cursor()
cur.execute("INSERT INTO utenti (nome) VALUES (%s)", ("Luca",))
conn.commit()
```

Query in PostgreSQL

Dopo l'inserimento, si recuperano i dati tramite `SELECT`. Il cursore fornisce tuple Python.

Le connessioni devono essere sempre chiuse per evitare perdite di risorse.

```
cur.execute("SELECT * FROM utenti")
for row in cur.fetchall():
    print(row)
cur.close()
conn.close()
```

ORM con SQLAlchemy – Definizione Modelli

SQLAlchemy ORM consente di lavorare con oggetti Python anziché scrivere SQL.

Si definiscono classi modello che mappano tabelle e colonne, mantenendo un approccio dichiarativo.

```
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy import Column, Integer, String, create_engine

Base = declarative_base()
class Utente(Base):
    __tablename__ = "utenti"
    id = Column(Integer, primary_key=True)
```

```
nome = Column(String)
```

```
engine = create_engine("sqlite:///db.sqlite")  
Base.metadata.create_all(engine)
```

ORM con SQLAlchemy – Operazioni CRUD

Le **sessioni** permettono di creare, leggere, modificare e cancellare record tramite oggetti.

Ogni oggetto rappresenta una riga nel database.

```
from sqlalchemy.orm import sessionmaker  
Session = sessionmaker(bind=engine)  
session = Session()  
session.add(Utente(nome="Anna"))  
session.commit()  
for u in session.query(Utente).all():  
    print(u.id, u.nome)
```

ORM con SQLAlchemy – Filtri e Query

È possibile costruire query dinamiche usando metodi fluenti (`filter` , `order_by`).

SQLAlchemy traduce automaticamente in SQL ottimizzato.

```
utenti = session.query(Utente).filter(Utente.nome.like("A%")).all()  
print([u.nome for u in utenti])
```

Lavorare con NoSQL – Introduzione

I database **NoSQL** gestiscono dati non strutturati (documenti, chiavi-valori, grafi).

Con Python, **MongoDB** è tra i più diffusi e si gestisce tramite la libreria `pymongo` .

```
from pymongo import MongoClient  
client = MongoClient("mongodb://localhost:27017")
```

```
db = client["azienda"]
db.utenti.insert_one({"nome": "Paolo", "ruolo": "Dev"})
```

Query in MongoDB

MongoDB utilizza query JSON-like per filtrare documenti.

I risultati sono cursori iterabili di dizionari Python.

```
for utente in db.utenti.find({"ruolo": "Dev"}):
    print(utente["nome"])
```

Logging in Python

Il modulo `logging` offre un sistema integrato di **tracciamento eventi**.

È utile per monitorare applicazioni e diagnosticare errori senza usare `print`.

```
import logging
logging.basicConfig(level=logging.INFO)
logging.info("Avvio applicazione")
logging.error("Errore di connessione")
```

Logger Personalizzati

È possibile creare logger con nomi e formati personalizzati.

I log possono essere inviati a file, console o servizi esterni.

```
logger = logging.getLogger("mio_logger")
handler = logging.FileHandler("app.log")
formatter = logging.Formatter("%(asctime)s - %(levelname)s - %(message)s")
handler.setFormatter(formatter)
logger.addHandler(handler)
logger.warning("Attenzione!")
```

Gestione Eccezioni Avanzata

Oltre ai blocchi `try/except`, Python consente gestione granulare di più eccezioni. Si possono catturare tipi specifici e accedere agli oggetti di errore.

```
try:
    x = 1 / 0
except ZeroDivisionError as e:
    print("Errore:", e)
```

Eccezioni Personalizzate

È possibile definire **eccezioni custom** derivando da `Exception`. Questo migliora la leggibilità e il debugging.

```
class InvalidAgeError(Exception):
    pass

def check_age(age):
    if age < 0:
        raise InvalidAgeError("Età non valida")

check_age(-2)
```

Moduli e Pacchetti

Un **modulo** è un file `.py`, mentre un **pacchetto** è una directory con `__init__.py`. Strutturare il codice in pacchetti migliora la riusabilità e l'organizzazione.

```
# struttura:
# mio_pacchetto/
# __init__.py
# utils.py

# utils.py
def saluta(): print("Ciao!")
```

Import Avanzato

Gli import possono essere **relativi** o **assoluti**.

Con `_all_`, si può controllare cosa viene esposto dal modulo.

```
from mio_pacchetto.utils import saluta
saluta()
```

Testing con unittest

Il modulo `unittest` permette test strutturati con setup, teardown e assert.

È la base della maggior parte dei framework di testing Python.

```
import unittest

class TestSomma(unittest.TestCase):
    def test_add(self):
        self.assertEqual(1 + 2, 3)

unittest.main()
```

Testing con pytest

pytest è un framework di test moderno e flessibile.

Offre sintassi semplice, fixture e ottimo output.

```
def test_add():
    assert 1 + 2 == 3
```

Mock e Patch nei Test

Per isolare dipendenze, `unittest.mock` consente di sostituire oggetti reali.

È utile nei test d'integrazione o API.

```
from unittest.mock import patch
```

```
@patch("modulo.esterno.chiamata_api", return_value={"ok": True})
def test_api(mock_api):
    assert mock_api() == {"ok": True}
```

Virtual Environment

Gli **ambienti virtuali** isolano le dipendenze di progetto.

Si creano con `venv` ed evitano conflitti tra pacchetti.

```
python -m venv venv
source venv/bin/activate # macOS/Linux
```

Decoratori – Concetto

I **decoratori** permettono di modificare funzioni o classi senza alterarne il codice.

Sono ampiamente usati per logging, validazione e caching.

```
def decoratore(fn):
    def wrapper():
        print("Inizio")
        fn()
        print("Fine")
    return wrapper

@decoratore
def saluta():
    print("Ciao!")

saluta()
```

Decoratori con Argomenti

Un decoratore può ricevere parametri definendo una funzione esterna.

È utile per configurazioni dinamiche.


```
def repeat(n):
    def decorator(fn):
        def wrapper():
            for _ in range(n):
                fn()
        return wrapper
    return decorator

@repeat(3)
def hello(): print("Hi!")

hello()
```

Generator

I **generatori** producono sequenze di valori al volo, risparmiando memoria.

Si definiscono con `yield` e sono iterabili.

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1
```

Coroutines e AsyncIO

Le **coroutine** permettono programmazione asincrona e non bloccante.

`asyncio` gestisce task concorrenti senza threading esplicito.

```
import asyncio

async def task():
    await asyncio.sleep(1)
    print("Fatto!")

asyncio.run(task())
```

Context Manager

I **context manager** gestiscono risorse (file, connessioni) in modo sicuro.

Definendo `__enter__` e `__exit__`, si può controllare il ciclo di vita di un oggetto.

```
class Risorsa:
    def __enter__(self): print("Apro"); return self
    def __exit__(self, *args): print("Chiudo")

with Risorsa():
    print("Uso la risorsa")
```

Typing e Tipizzazione Statica

Il modulo `typing` consente di annotare tipi per migliore documentazione e linting.

Le annotazioni non sono vincolanti ma aiutano strumenti come mypy.

```
def somma(a: int, b: int) → int:
    return a + b
```

Type Alias e Generics

È possibile definire **alias di tipo** e generici per funzioni generali.

Questo aumenta la chiarezza del codice.

```
from typing import List, TypeVar

T = TypeVar("T")
def prima(lista: List[T]) → T:
    return lista[0]
```

Profilazione

Per ottimizzare il codice si usa `cProfile` o `timeit`.

Questi strumenti identificano i colli di bottiglia.

```
import cProfile
cProfile.run("sum(range(1000000))")
```

Benchmark con timeit

`timeit` misura l'esecuzione media di un'espressione Python.

Utile per confrontare diverse implementazioni.

```
import timeit
print(timeit.timeit("sum(range(1000))", number=1000))
```

Observer Pattern

Il **pattern Observer** notifica automaticamente gli oggetti interessati ai cambiamenti di stato.

È utile per sistemi di eventi o UI reattive.

```
class Subject:
    def __init__(self): self._obs = []
    def attach(self, o): self._obs.append(o)
    def notify(self, msg): [o.update(msg) for o in self._obs]

class Observer:
    def update(self, msg): print("Ricevuto:", msg)
```

Decorator Pattern

Permette di **aggiungere comportamenti** a oggetti dinamicamente senza modificarne la classe.

Si usa per logging, caching o validazione.

```
class Messaggio:
    def mostra(self): print("Ciao")

class Decoratore:
```

```
def __init__(self, msg): self._msg = msg
def mostra(self):
    print("Decorato →", end=" ")
    self._msg.mostra()
```

```
Decoratore(Messaggio()).mostra()
```

Adapter Pattern

Adatta interfacce incompatibili traducendo chiamate tra classi.

Consente di riutilizzare codice esistente senza modificarlo.

```
class VecchioSistema:
    def esegui(self): print("Sistema legacy")

class Adapter:
    def __init__(self, target): self.target = target
    def run(self): self.target.esegui()
```

Template Method Pattern

Definisce lo **scheletro di un algoritmo**, delegando i passi a sottoclassi.

Garantisce struttura e riuso del flusso.

```
from abc import ABC, abstractmethod
class Base(ABC):
    def process(self):
        self.step1(); self.step2()
    @abstractmethod
    def step1(self): ...
    @abstractmethod
    def step2(self): ...
```

Command Pattern

Incorpora le operazioni in oggetti, rendendo possibile annullarle o accodarle.

È comune in GUI e sistemi di undo/redo.

```
class Command:
    def __init__(self, action): self.action = action
    def execute(self): self.action()
```

Iterator Pattern

Consente di scorrere collezioni definite personalizzando `__iter__` e `__next__`.

È la base di generatori e loop `for`.

```
class Contatore:
    def __init__(self, n): self.n = n
    def __iter__(self): return self
    def __next__(self):
        if self.n <= 0: raise StopIteration
        self.n -= 1
        return self.n
```

State Pattern

Permette di cambiare comportamento di un oggetto modificandone lo stato interno.

Evita catene di `if/else` basate su flag.

```
class Stato:
    def handle(self): pass

class On(Stato):
    def handle(self): print("Acceso")

class Off(Stato):
    def handle(self): print("Spento")
```

Context Pattern

Combina più pattern per gestire comportamenti dipendenti dal contesto.

Può essere visto come un'evoluzione del State.

```
class Context:
    def __init__(self, state): self.state = state
    def request(self): self.state.handle()
```

Proxy Pattern

Il **Proxy** controlla l'accesso a un oggetto reale, aggiungendo caching, logging o sicurezza.

```
class Proxy:
    def __init__(self, real): self.real = real
    def action(self):
        print("Controllo accesso"); self.real.action()
```

Singleton Pattern

Garantisce che una sola istanza di una classe esista nell'applicazione.

Utile per log centralizzati o configurazioni globali.

```
class Singleton:
    _instance = None
    def __new__(cls):
        if not cls._instance:
            cls._instance = super().__new__(cls)
        return cls._instance
```

Factory Pattern

Crea oggetti senza esporre la logica di costruzione.

Riduce la dipendenza dal tipo concreto.

```
class Factory:
    @staticmethod
```

```
def crea(tipo):  
    if tipo == "A": return list()  
    if tipo == "B": return dict()
```

Strategy Pattern

Definisce una **famiglia di algoritmi intercambiabili**.

Consente di modificare il comportamento a runtime.

```
def upper(t): return t.upper()  
def lower(t): return t.lower()  
  
class Text:  
    def __init__(self, strategy): self.strategy = strategy  
    def show(self, s): print(self.strategy(s))
```